

Node.js and Express.js

- **transpiled** ==> *Translate + Compile* = Transpile (Source-to-Source Compiler).
- Transpiling means taking source code written in one version/syntax of a language and translating it into an older or standard version of the SAME language so that older software can understand it.
- Real-World Examples in Web Development:
 - Transpiling JSX ==> Plain JavaScript
 - Web browsers do NOT understand JSX (like

Hello {name}

).

- A transpiler (**like *Babel* or *Vite***) takes your JSX and translates (transpiles) it into standard JavaScript:

```
// What you write:
<h1>Hello {name}</h1>

// What the transpiler turns it into for the browser:
React.createElement('h1', null, `Hello ${name}`)
```

- Transpiling Modern JS (ES2024) ==> Older JS (ES5)
- Transpiling vs Compiling (The Difference):

Compiling: Taking human code (like C++ or Rust) and *turning it into Machine Code / 0s and 1s (.exe)*.

Transpiling: Taking human code (like JSX or TypeScript) and *turning it into another human code (Standard JavaScript)*!

- `npm init -y` (skips all que's) to initialize the **package.json**, here I am building a backend from **bare-metal scratch** with **no templates**!
- then create `index.js` and **Configure package.json** with **"start": "node index.js"**.

Simple web server

- First Server :-

```
const http = require('http')
```

```
const app = http.createServer((request, response) => {
  response.writeHead(200, { 'Content-Type': 'text/plain' })
  response.end('Hello World')
})

const PORT = 3001
app.listen(PORT)
console.log(`Server running on port ${PORT}`)
```

1. const http = require('http')

1. http is a built-in module inside Node.js.

2. **require() is the older Node.js** way of importing packages (**it's the exact same concept as import http from 'http'**).

2. The (request, response) Callback

```
http.createServer((request, response) => { ... })
```

- Every web server in the world operates on *Request* and *Response*:
 - **request**: Contains everything the user sent (the **URL** they clicked, **the data**, **headers**).
 - **response**: The *pipe you use to send data BACK to the user's browser*.
 - response.writeHead(200, { 'Content-Type': 'text/plain' }): Tells the browser: "Status is 200 OK, and I am sending you plain text."
 - response.end('Hello World'): Sends "Hello World" and closes the connection!
-

3. app.listen(PORT) (The Doorbell)

```
const PORT = 3001
app.listen(PORT)
```

- this is just basic understanding of how we make server live. but very traditional method
-

Express-Js

- Express is a lightweight library that removes 90% of the painful boilerplate of Node's raw http module.
- **Transitive Dependencies** ((The dependencies of your dependencies!)) :

- The creators of Express didn't write every single piece of code from scratch. Express relies on smaller helper packages (like cookie, accepts, content-type, qs). And those helper packages rely on other smaller helper packages!
 - Your Project ==> Express ==> helper-package ==> sub-helper-package
 - semantic versioning
 - Caret symbol ^
 - Major . Minor . Patch
 - understand by e.g.
 - Major - 4.18.2 ==> 5.0.0
 - Minor - 4.18.2 ==> 4.19.0
 - Patch - 4.18.2 ==> 4.18.3
-

Web and Express

- First Server with **Express**:

```
const express = require('express'); // import express from 'express';
const app = express();

let notes = [
  {
    id: "1",
    content: "HTML is easy",
    important: true
  },
  {
    id: "2",
    content: "Browser can execute only JavaScript",
    important: false
  },
  {
    id: "3",
    content: "GET and POST are the most important methods of HTTP protocol",
    important: true
  }
];

app.get('/', (request, response) => {
  response.send('<h1>Hello Vinayak, Welcome to NodeJS World!</h1>');
});

app.get('/api/notes', (request, response) => {
  response.json(notes)
});
```

```
const PORT = 3001;

app.listen(PORT, () => {
  console.log(`Server Running on Port : http://localhost:${PORT}`)
});
```

- we define two routes to the application.
 - The first one defines an event handler that is used to handle *HTTP GET requests made to the application's / root*:

```
app.get('/', (request, response) => {
  response.send('<h1>Hello World!</h1>')
})
```

- The event handler function accepts **two parameters**.
 - The first request parameter contains all of the information of the HTTP request and the second response parameter is used to define how the request is responded to.
 - In our code, the request is answered by using the **send** method of the **response object**.
 - Express automatically sets the value of the Content-Type header to be text/html.
 - content-type :- **text/html**; charset=utf-8
 - The status code of the response defaults to 200.

-
- The second route defines an event handler that handles HTTP GET requests made to the notes path of the application:

```
app.get('/api/notes', (request, response) => {
  response.json(notes)
})
```

- The request is responded to with the *json* method of the **response object**.
 - Calling the method will send the notes array that was passed to it as a JSON formatted string.
 - Express automatically sets the **Content-Type header** with the appropriate value of **application/json**.
 - content-type :- **application/json**; charset=utf-8

-
- In the earlier version where we were only using Node, we had to transform the data into the JSON formatted string with the *JSON.stringify* method:

```
response.end(JSON.stringify(notes))
```

- With Express, this is no longer required, because this transformation happens automatically.
 - It's worth noting that JSON is a data format. However, it's often represented as a string and is not the same as a JavaScript object, like the value assigned to notes.
- `JavaScript Object` vs `JSON.stringify(object)`

```
'JavaScript Object' typeOf = object
const person = { name: "vinayak", age: 23 }

const json = JSON.stringify(person);
'json is become now json string object and it will return everything in
String format'

return {"name":"vinayak","age":23}
'JSON String Object' typeOf = string
```

Automatic Change Tracking

- kill old session of the server and restart the server on the change of single line of code into `index.js`
 - configure script in **package.json**
 - **"dev":"node --watch index.js"**
 - and shoot `npm run dev`

REST - Representational State Transfer, aka REST

- Let's expand our application so that it provides the same RESTful HTTP API as `json-server`
- If we define the resource type of note to be *notes*, then the address of a note resource with the identifier 10, has the unique address **`www.example.com/api/notes/10`**.
- The URL for the entire collection of all note resources is **`www.example.com/api/notes`**.
- We can execute different operations on resources. The operation to be executed is defined by the HTTP verb:

URL	verb	functionality
notes/10	Get	fetches a single resource
notes	Get	fetches all resources in the collection

URL	verb	functionality
notes	Post	creates a new resource based on the request data
notes/10	Delete	removes the identified resource
notes/10	Put	replaces the entire identified resource with the request data
notes/10	Patch	replaces a part of the identified resource with the request data

Fetching a single resource

```
app.get('/api/notes/:id', (request, response) => {  
  const id = request.params.id  
  const note = notes.find(note => note.id === id)  
  response.json(note)  
})
```

- Now `app.get('/api/notes/:id', ...)` will handle all HTTP GET requests that are of the form `/api/notes/SOMETHING`, where `SOMETHING` is an arbitrary string.
- If we search for a note with an id that does not exist, the server responds with:
 - `'/api/notes/100/'`
 - The HTTP status code that is returned is 200, which means that the response succeeded. There is no data sent back with the response, since the value of the content-length header is 0, and the same can be verified from the browser.
 - The reason for this behavior is that the `note` variable is set to `undefined` if no matching note is found.
 - The situation needs to be handled on the server in a better way. If no note is found, the server should respond with the status code 404 not found instead of 200.

```
app.get('/api/notes/:id', (request, response) => {  
  const id = request.params.id  
  const note = notes.find(note => note.id === id)  
  if(!note){  
    return response.status(404).end()  
  }  
  response.json(note)  
})
```

- **The Golden Rule of Express Guard Clauses:**
 - In Express, `response.send()` / `response.end()` does not stop JavaScript. Always write **return response.status(...)** to exit early!

- Since no data is attached to the response, **we use the status method** for **setting the status** and the **end method for responding to the request without sending any data**.

Deleting resources

- Next, let's implement a route for deleting resources. Deletion happens by making an HTTP DELETE request to the URL of the resource:

```
app.delete('/api/notes/:id', (request, response) => {  
  const id = request.params.id  
  notes = notes.filter(note => note.id !== id)  
  
  response.status(204).end()  
})
```

- If deleting the resource is successful, meaning that the note exists and is removed, we respond to the request with the **status code 204 no content** and **return no data with the response**.
- There's no consensus on what status code should be returned to a DELETE request if the resource does not exist. **The only two options are 204 and 404**. For the sake of simplicity, our application will respond with 204 in both cases.

Receiving data

- Next, let's make it possible to add new notes to the server.
- Adding a note happens by making an **HTTP POST request** to the address `http://localhost:3001/api/notes`.
- and by sending all the information for the new note in the **request body in JSON format**.
- Process to activate the json-parser and implement an initial handler for dealing with the HTTP POST requests:

```
const express = require('express')  
const app = express()  
  
<!-- Middleware -->  
app.use(express.json())  
  
//...  
  
app.post('/api/notes', (request, response) => {  
  const note = request.body  
  console.log(note)
```

```
response.json(note)
})
```

- Why it is so important:
 - When a client sends JSON in a POST request, it arrives at the server as a **raw stream of text**.
 - `app.use(express.json())` is Express's built-in JSON Parser Middleware.
 - **It intercepts the incoming raw text**,** *parses it into a real JavaScript object***, and *attaches it to `request.body`*!
 - ⚠ If you forget `app.use(express.json())`, **`request.body` will always be undefined!**

Middleware

- **`express.json()` IS a middleware!** Whenever a **request arrives**, **`express.json()` intercepts it**, **parses the raw text into a JS object**, and **passes it forward to your `app.post()` route!**

About HTTP request types

- The HTTP standard talks about two properties related to request types, **safety** and **idempotency**.
- **Safety** means that the executing request must not ***cause any side effects*** on the **server**.
- By side effects, we mean that **the state of the database must not change as a result of the request**, and the response must only return data that already exists on the server.
- **`Request(browser) => backend(processing request.body from Safety measures)`**
 - ==> **`if(Found threats or irrelevant stuff)`** ==> handle it on server, response back to **browser** with appropriate **Error**
 - ==> **`else(Heads towards database)`**
- **"Safe Methods" :**
 - A "Safe" method is a request that **ONLY READS** data and **NEVER** modifies or destroys anything on the server.
 - GET is a Safe method because reading data never changes the database.
- **"Idempotent Methods"**
 - **"Idempotent"** (pronounced eye-dem-po-tent) means: Running a request 1 time produces the **EXACT SAME** result as running it 100 times in a row!
 - Let's look at the Elevator Button vs ATM Button analogy:
 - The Elevator Button (*Idempotent*):

- If you press the elevator button 1 time, the elevator comes to your floor. If you aggressively press the elevator button 50 times in a row, **the elevator still just comes to your floor! The end result is identical.**
 - The ATM Cash Button (*Non-Idempotent / POST*):
 - If you press "Withdraw \$100" 1 time, you get \$100. If you press "Withdraw \$100" 5 times, you just withdrew \$500 and your bank account is empty!
 - **GET is Idempotent:** Fetching data 10 times gives the same result.
 - **PUT is Idempotent:** If you update Arto's number to "123-456", running that exact update 10 times still leaves his number as "123-456".
 - **DELETE is Idempotent:** Deleting Note #3 leaves Note #3 deleted. Deleting it 5 more times still means Note #3 is deleted.
 - **POST is NOT Idempotent:** If you send a POST request to create "Arto" 5 times, the server will create 5 separate duplicate records with 5 new IDs!
-

Middleware FSO

- The Express json-parser used earlier is a middleware.
- Middleware are functions that can be used for handling request and response objects.
- In practice, you can use several middlewares at the same time. When you have more than one, they're executed one by one in the order that they were listed in the application code.
- Middleware is a function that receives three parameters:

```
const requestLogger = (request, response, next) => {  
  console.log('Method:', request.method)  
  console.log('Path: ', request.path)  
  console.log('Body: ', request.body)  
  console.log('---')  
  next() <===  
}
```

- At the end of the function body, the next function that was passed as a parameter is called **next**. The next function **yields control to the next middleware**.
- Middleware is used like this:

```
app.use(express.json())  
app.use(requestLogger)
```

- Middleware functions have to be used before routes when we want them to be executed by the route event handlers.
- Sometimes, we want to use middleware functions after routes. We do this when the middleware functions are only called if no route handler processes the HTTP request.
 - This middleware will be used for catching requests made to non-existent routes

```
const unknownEndpoint = (request, response) => {
  response.status(404).send({ error: 'unknown endpoint' })
}

app.use(unknownEndpoint)
```

Middleware by Mentor

- Imagine an Express server as a Factory **Conveyor Belt** => automatic vertical elevator/pusher
- Whenever an **HTTP request arrives from a browser or Postman, it is placed** on the **conveyor belt** and **passes through** several Stations (**Middleware Functions**) in order from top to bottom:

```
[ Request Arrives ]
  ↓

[ Station 1: express.json() ] --> (Parses raw text into request.body)
  ↓ calls next()

[ Station 2: Morgan Logger ] --> (Logs request details to terminal)
  ↓ calls next()

[ Station 3: Your Routes ] --> (app.get, app.post, app.delete)
  ↓ (If no route matched)

[ Station 4: Unknown Endpoint ] --> (Returns 404 error)
```

- Why Middleware Functions have 3 Parameters: (request, response, next)
 - Every middleware function in Express looks like this:

```
const myMiddleware = (request, response, next) => {
  console.log('A request passed through Station 1!');

  // 🚨 CRITICAL: You MUST call next()!
  next();
};

app.use(myMiddleware);
```

- What is next()?
 - next() is the "Push Conveyor Belt Forward" button!
 - Why the Order of Middleware Matters:
 - **express.json() and loggers must be at the TOP** : They need to parse and log the request before it reaches your routes.
 - **unknownEndpoint (404 catch-all) must be at the VERY BOTTOM** : If you put the 404 handler at the top, it will intercept and block every single request before your routes even have a chance to run!
-

Morgan - Logger

basic setup of morgan logger

- **npm install morgan**
- `const morgan = require('morgan');`
 - `// Use the pre-built 'tiny' format app.use(morgan('tiny'));`
- What it does:
 - Whenever you visit `http://localhost:3001/api/persons`, Morgan instantly prints this in your terminal:
 - `GET /api/persons 200 245 - 2.154 ms`
 - (Method(GET), URL(/api/persons), Status Code(200), Response Size(245), Time taken in milliseconds) - (2.154ms).
 -

Custom Morgan Token (Logging the POST Body!)

- By default, Morgan does not log the request body. But Morgan lets you create a Custom Token using **morgan.token()**!
 - How to create the custom token:

1. Create a token called 'body' that stringifies request.body:

```
// Creates a custom Morgan token named :body
morgan.token('body', (request, response) => {
  return JSON.stringify(request.body);
});
```

2. Configure Morgan using a custom format string that includes :body:

```
app.use(morgan(':method :url :status :res[content-length] -  
:response-time ms :body') );
```

- The Result in your Terminal:
 - When you send a **GET /api/persons**, it prints: **GET /api/persons 200 245 - 2.154 ms {}**
 - When you send a POST /api/persons with a new contact, it prints: **POST /api/persons 200 64 - 4.231 ms {"name":"Liisa Marttinen","number":"040-243563"}!**